

# SHETH LUJ & M V COLLEGE

## PRACTICAL NO 2

AIM :

Implement the A\* Search algorithm for solving a pathfinding problem.

Implement the Recursive Best-First Search algorithm for the same problem.

Compare the performance and effectiveness of both algorithms.

### 1.) ROCKTHEIT :

```
dict_hn = {
    'MVLU College': 15,
    'Malad': 10,
    'Kandivali': 8,
    'Borivali': 4,
    'Sanjay Gandhi National Park': 0
}

dict_gn = dict(
    **{
        'MVLU College': dict(Malad=6, Kandivali=8),
        'Malad': dict(MVLU_College=6, Borivali=5),
        'Kandivali': dict(MVLU_College=8, Borivali=4),
        'Borivali': dict(Malad=5, Kandivali=4, **{'Sanjay Gandhi National
Park':3}),
        'Sanjay Gandhi National Park': dict()
    }
)

# Rename key to match dictionary
dict_gn['Malad']['MVLU College'] = dict_gn['Malad'].pop('MVLU_College')
dict_gn['Kandivali']['MVLU College'] =
dict_gn['Kandivali'].pop('MVLU_College')

import queue as Q

start = 'MVLU College'
goal = 'Sanjay Gandhi National Park'
```

## SHETH LUJ & M V COLLEGE

```
result = "
```

```
def get_fn(citystr):  
    cities = citystr.split(" , ")  
    hn = gn = 0  
  
    for ctr in range(len(cities)-1):  
        gn += dict_gn[cities[ctr]][cities[ctr+1]]  
  
    hn = dict_hn[cities[-1]]  
    return hn + gn
```

```
def expand(cityq):  
    global result  
  
    tot, citystr, thiscity = cityq.get()  
  
    if thiscity == goal:  
        result = citystr + " : : " + str(tot)  
        return  
  
    for cty in dict_gn[thiscity]:  
        cityq.put((get_fn(citystr + " , " + cty),  
                  citystr + " , " + cty,  
                  cty))  
  
    expand(cityq)
```

```
def main():  
    cityq = Q.PriorityQueue()  
  
    cityq.put((get_fn(start), start, start))
```

## SHETH LUJ & M V COLLEGE

```
expand(cityq)
```

```
print("The A* path with the total cost is:")  
print(result)
```

```
main()
```

### OUTPUT :

```
===== RESTART: C:/Users/mvluc/Desktop/tycs11/PRAC2.py =====  
The A* path with the total cost is:  
MVLU College , Kandivali , Borivali , Sanjay Gandhi National Park : : 15  
>
```

## 2.) A\* SEARCH ALGORITHM :

```
from queue import PriorityQueue
```

```
# A = Andheri
```

```
# B = Jogeshwari
```

```
# C = Bandra
```

```
# D = Mahim
```

```
# E = Dadar
```

```
# F = Worli
```

```
# G = Churchgate (Goal)
```

```
graph = {  
    'A': {'B': 4, 'C': 8},  
    'B': {'D': 6},  
    'C': {'D': 3},  
    'D': {'E': 4},  
    'E': {'F': 5},  
    'F': {'G': 6},  
    'G': {}  
}
```

```
heuristic = {  
    'A': 15,  
    'B': 13,  
    'C': 10,  
    'D': 8,  
    'E': 5,  
    'F': 3,  
    'G': 0  
}
```

```
start = 'A'
```

```
goal = 'G'
```

## SHETH LUJ & M V COLLEGE

```
pq = PriorityQueue()
pq.put((heuristic[start], 0, [start])) # (f, g, path)

visited = set()

while not pq.empty():

    f, g, path = pq.get()
    node = path[-1]

    if node == goal:
        print("\nBest Path:", " -> ".join(path))
        print("Total Cost =", g)
        break

    if node in visited:
        continue

    visited.add(node)

    for neighbour, cost in graph[node].items():
        new_g = g + cost
        new_f = new_g + heuristic[neighbour]

        print(path + [neighbour],
              "g =", new_g,
              "h =", heuristic[neighbour],
              "f =", new_f)

        pq.put((new_f, new_g, path + [neighbour]))
```

# SHETH LUJ & M V COLLEGE

## OUTPUT :

```
>>> Enter help below or click help above for more information.
>>> ===== RESTART: C:/Users/mvluc/Desktop/tycs11/PRAC2.py =====
['A', 'B'] g = 4 h = 13 f = 17
['A', 'C'] g = 8 h = 10 f = 18
['A', 'B', 'D'] g = 10 h = 8 f = 18
['A', 'C', 'D'] g = 11 h = 8 f = 19
['A', 'B', 'D', 'E'] g = 14 h = 5 f = 19
['A', 'B', 'D', 'E', 'F'] g = 19 h = 3 f = 22
['A', 'B', 'D', 'E', 'F', 'G'] g = 25 h = 0 f = 25

Best Path: A -> B -> D -> E -> F -> G
Total Cost = 25
>>>
```

### 3.) A\* SEARCH ALGORITHM MAP PLOT :

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

# -----
# 1. DATA STRUCTURE
# -----

graph = {
    'MVLU College': {'Malad': 6, 'Kandivali': 8},
    'Malad': {'Borivali': 5},
    'Kandivali': {'Borivali': 4},
    'Borivali': {'Sanjay Gandhi National Park': 3},
    'Sanjay Gandhi National Park': {}
}

heuristics = {
    'MVLU College': 15,
    'Malad': 10,
    'Kandivali': 8,
    'Borivali': 4,
    'Sanjay Gandhi National Park': 0
}

node_positions = {
    'MVLU College': (1, 5),
    'Malad': (2, 4),
    'Kandivali': (2, 6),
    'Borivali': (4, 5),
    'Sanjay Gandhi National Park': (6, 5)
}
```

## SHETH LUJ & M V COLLEGE

```
# -----  
# 2. A* SEARCH ALGORITHM  
# -----
```

```
def a_star_search(graph, heuristics, start, goal):  
  
    priority_queue = [(heuristics[start], start, [start], 0)]  
    visited = set()  
  
    while priority_queue:  
  
        f_score, current, path, g_score = heapq.heappop(priority_queue)  
  
        if current in visited:  
            continue  
  
        visited.add(current)  
  
        if current == goal:  
            return path, g_score  
  
        for neighbor, distance in graph[current].items():  
  
            if neighbor not in visited:  
  
                new_g = g_score + distance  
                new_f = new_g + heuristics[neighbor]  
  
                heapq.heappush(  
                    priority_queue,  
                    (new_f, neighbor, path + [neighbor], new_g)  
                )  
  
    return None, float('inf')
```



## SHETH LUJ & M V COLLEGE

```
# -----
```

```
# 3. RUN A* SEARCH
```

```
# -----
```

```
optimal_path, total_distance = a_star_search(  
    graph,  
    heuristics,  
    'MVLU College',  
    'Sanjay Gandhi National Park'  
)
```

```
print("Optimal Path Discovered:")  
print(" -> ".join(optimal_path))  
print("Total Road Distance =", total_distance, "km")
```

```
# -----
```

```
# 4. GRAPH VISUALIZATION
```

```
# -----
```

```
G = nx.DiGraph()
```

```
for node, neighbours in graph.items():  
    for neighbour, distance in neighbours.items():  
        G.add_edge(node, neighbour, weight=distance)
```

```
plt.figure(figsize=(10,7))
```

```
path_edges = list(zip(optimal_path, optimal_path[1:]))  
normal_edges = [edge for edge in G.edges() if edge not in path_edges]
```

```
nx.draw_networkx_nodes(  
    G,  
    node_positions,
```

## SHETH LUJ & M V COLLEGE

```
node_size=2200,
node_color='lightblue'
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    edge_color='gray',
    width=1.5,
    arrows=True
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    edge_color='orange',
    width=3,
    arrows=True
)

labels = {
    node: f"{node}\nh={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=labels,
    font_size=8,
    font_weight='bold'
)
```

## SHETH LUJ & M V COLLEGE

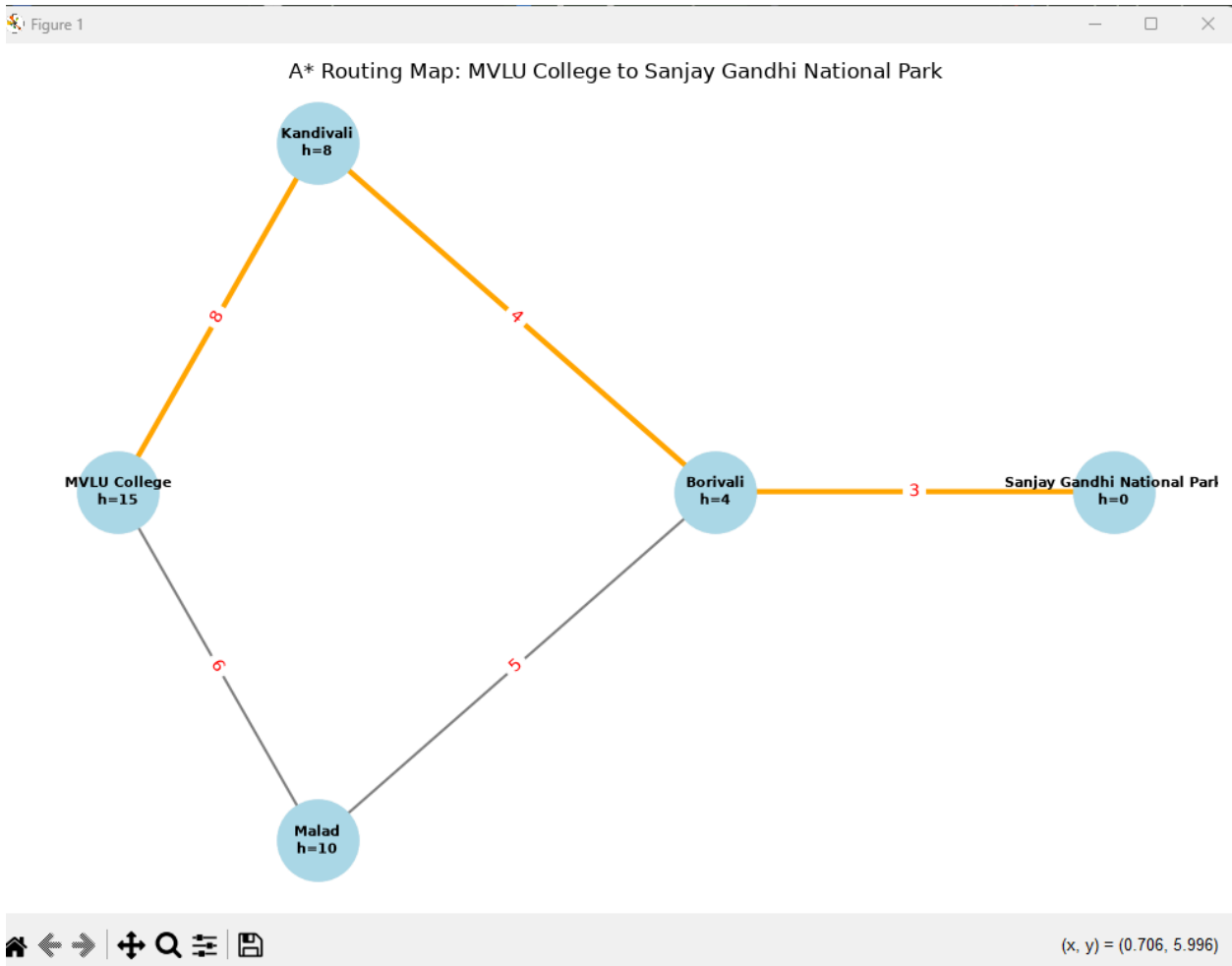
```
edge_labels = nx.get_edge_attributes(G, 'weight')

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color='red'
)

plt.title("A* Routing Map: MVLU College to Sanjay Gandhi National Park")
plt.axis("off")
plt.tight_layout()
plt.show()
```

# SHETH LUJ & M V COLLEGE

OUTPUT :



## SHETH LUJ & M V COLLEGE

### 4.) RECURSIVE :

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# 1. NETWORK DEFINITION
```

```
# -----
```

```
graph = {  
    'MVLU College': {'Malad': 6, 'Kandivali': 8},  
    'Malad': {'Borivali': 5},  
    'Kandivali': {'Borivali': 4},  
    'Borivali': {'Sanjay Gandhi National Park': 3},  
    'Sanjay Gandhi National Park': {}  
}
```

```
heuristics = {  
    'MVLU College': 15,  
    'Malad': 10,  
    'Kandivali': 8,  
    'Borivali': 4,  
    'Sanjay Gandhi National Park': 0  
}
```

```
# Coordinates for plotting
```

```
coords = {  
    'MVLU College': (1, 5),  
    'Malad': (2, 4),  
    'Kandivali': (2, 6),  
    'Borivali': (4, 5),  
    'Sanjay Gandhi National Park': (6, 5)  
}
```

```
# -----
```

## SHETH LUJ & M V COLLEGE

### # 2. RECURSIVE BEST-FIRST SEARCH

# -----

```
def rbfs_search(start, goal):  
    success, path, cost, _ = rbfs(start, goal, 0, float('inf'), [start])  
    return path, cost
```

```
def rbfs(node, goal, g, f_limit, path):  
  
    if node == goal:  
        return True, path, g, g  
  
    neighbours = graph[node]  
  
    if not neighbours:  
        return False, [], 0, float('inf')  
  
    successors = []  
  
    for neighbour, distance in neighbours.items():  
  
        if neighbour not in path:  
  
            new_g = g + distance  
            new_f = max(new_g + heuristics[neighbour],  
                        g + heuristics[node])  
  
            successors.append([new_f, neighbour, new_g])  
  
    if not successors:  
        return False, [], 0, float('inf')  
  
    while True:
```

## SHETH LUJ & M V COLLEGE

```
successors.sort(key=lambda x: x[0])

best = successors[0]

if best[0] > f_limit:
    return False, [], 0, best[0]

alternative = successors[1][0] if len(successors) > 1 else float('inf')

success, result_path, total_g, returned_f = rbfs(
    best[1],
    goal,
    best[2],
    min(f_limit, alternative),
    path + [best[1]]
)

best[0] = returned_f

if success:
    return True, result_path, total_g, returned_f

# -----
# 3. RUN RBFS
# -----

path, total_dist = rbfs_search('MVLU College', 'Sanjay Gandhi National Park')

print("Optimal RBFS Path:")
print(" -> ".join(path))

print("Total Distance =", total_dist, "km")
```

## SHETH LUJ & M V COLLEGE

```
# -----
```

```
# 4. GRAPH PLOTTING
```

```
# -----
```

```
plt.figure(figsize=(8,6))
```

```
for node, neighbours in graph.items():
```

```
    x1, y1 = coords[node]
```

```
    for neighbour, dist in neighbours.items():
```

```
        x2, y2 = coords[neighbour]
```

```
        is_path = (  
            node in path and  
            neighbour in path and  
            path.index(neighbour) == path.index(node) + 1  
        )
```

```
        if is_path:  
            color = "green"  
            width = 3
```

```
        else:  
            color = "gray"  
            width = 1.5
```

```
        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width)  
        plt.text((x1+x2)/2, (y1+y2)/2, str(dist) + " km", color="red")
```

```
for node, (x, y) in coords.items():
```

```
    c = "yellow" if node in path else "skyblue"
```



## SHETH LUJ & M V COLLEGE

```
plt.scatter(x, y, s=700, color=c)
```

```
plt.text(x, y,  
        node + "\nh=" + str(heuristics[node]),  
        ha="center",  
        va="center",  
        fontsize=8)
```

```
plt.title("RBFS : MVLU College to Sanjay Gandhi National Park")  
plt.axis("off")  
plt.show()
```

# SHETH LUJ & M V COLLEGE

OUTPUT :

